



# Fix the Code

Find the wrong block and fix it

Name: .....

Date: .....

★ I can find the wrong block in a code and fix it.

## This code has a bug

This code is supposed to land Bit on 4, but one block is WRONG, so Bit misses it. A coder runs the code, finds the wrong block, and fixes it. Bit starts on 1. The target is 4.

## Bit's number path (target is 4)

|   |   |   |   |             |   |
|---|---|---|---|-------------|---|
| 0 | 1 | 2 | 3 | 4<br>target | 5 |
|---|---|---|---|-------------|---|

## Buggy code — should reach 4

start at 1

forward 2

back 1

### 1 Run it

Run the buggy code from 1: forward 2, then back 1. What number does Bit land on? Is it the target (4)?

### 2 Find the wrong block

One block moves Bit the WRONG way and sends it away from 4. Circle the block that is wrong.

---

## Fixed code

start at 1

forward 2



*Write the block that makes Bit reach 4.*

### 3 You try — fix it

Write the correct block in the empty space so Bit reaches 4. Then run the fixed code. What number does Bit land on now?

## Big idea

Finding and fixing a wrong block is called debugging. Run the code, see Bit miss the target, find the block that is wrong, fix it, and run it again.



# Fix the Code

Quick facilitation notes & answer key

## What this worksheet teaches

Students debug a set of instructions: run it, see Bit miss the target, find the one wrong block, fix it, and run again to check. Run it, find the wrong block, fix it, run again — a first debugging routine. No coding experience needed.

## Before you start (no computer needed)

Paper and pencil only. Optional: a floor number line 0 to 5 with a marker on 4 (the target) so students can walk the buggy code and feel Bit go the wrong way.

## How to lead it (about 20 minutes)

1. Show them first (you do). Run the buggy code: "start 1... forward 2 to 3... back 1 to 2. Bit is on 2, but the target is 4 — something is wrong."
2. Do one together (we do). Exercise 1: the buggy code lands on 2 (not 4). Exercise 2: the back 1 block moves Bit away from 4 — circle it.
3. Let them try alone (you do). Exercise 3: replace the wrong block with forward 1; the code becomes start 1, forward 2, forward 1 and reaches 4.
4. Big idea: "Run it, find the wrong block, fix it, run again — that is how coders debug."

## Answer key (these are the verified correct answers)

Exercise 1 — buggy code: 1 → forward 2 → 3 → back 1 → 2. Bit lands on 2, not the target 4.

Exercise 2 — the wrong block is back 1: it sends Bit backward, away from 4.

Exercise 3 — fix it to forward 1: 1 → 3 → 4. Bit lands on the target, 4.

Why this is the bug: the first two blocks bring Bit to 3, one short of 4; a forward 1 finishes the job, but a back 1 undoes progress and lands on 2.

## If students get stuck — and ways to adjust

Watch for: blaming the start or the forward 2. Have students run step by step and watch exactly where Bit turns away from 4.

Make it easier: walk the buggy code on the floor line, then swap the last card to forward 1 and re-walk.

Make it harder: ask for a DIFFERENT single-block fix that also reaches 4 — e.g. change forward 2 to forward 4 and keep back 1: 1 → 5 → 4.

## Ontario curriculum link (official wording)

C3.1 — solve problems and create computational representations of mathematical situations by writing and executing code, including code that involves sequential events.

In plain terms: students follow (or write) step-by-step instructions, in order, to get a result.

C3.2 — read and alter existing code, including code that involves sequential events, and describe how changes to the code affect the outcomes.

In plain terms: students read instructions someone wrote, change one, and explain what the change did.

## You'll know it worked when

The student lands the buggy code on 2, identifies back 1 as the wrong block, and fixes it to forward 1 to reach 4.